NeuFlow v3

Queryable optical flow with calibrated uncertainty

Shriman Raghav Srinivasan

MS Robotics, Northeastern University · Field Robotics Lab · August 2026

Summary

NeuFlow v2 always computes flow for every pixel at one fixed resolution. v3 replaces only its final upsampling stage with an implicit decoder, so the model answers flow queries at any continuous coordinate. Cost scales with the number of points asked for, $O(N)$, not with image area, $O(H \times W)$.

The decoder costs accuracy: 2.384 px against v2's 2.324 px, so 2.6% worse on mean error and 1.5 points worse on 1-pixel accuracy. In exchange it adds three things v2 cannot do at all: flow at sub-pixel coordinates, repeat queries on a cached frame at 27× lower cost, and a calibrated confidence value per query, from a model with 13% fewer parameters. This report prices that trade.

1. Why queryable flow

Flow networks produce a dense map. Many consumers do not want one.

- Registration needs a few hundred correspondences at points it picks itself.
- Sparse tracking needs flow only at feature points.
- Survey mosaicking needs matches in overlap regions, often between pixels.

Computing 479,232 values to use 800 of them is the difference between real time and not on constrained hardware. A fixed-resolution design also cannot answer any query above its input sampling rate, which is what high-resolution registration needs.

The question this project asks: can the last stage be made queryable without losing accuracy or speed?

2. Method

2.1 What is inherited

NeuFlow v2 (Zhang, Gupta, Jiang & Singh, arXiv:2408.10161) runs a shallow CNN backbone at 1/8 and 1/16 resolution, cross-attention and global matching at 1/16 for an initial flow, recurrent refinement (one iteration at 1/16, eight at 1/8), then a learned convex upsampler to full resolution.

v3 keeps all of that up to the 1/8 coarse flow, with every learned weight frozen and every batch-normalisation running statistic held at v2's value. I verify this per checkpoint: all 137 shared tensors are bit-identical to v2, so the comparison below isolates the decoder and nothing else.

2.2 The decoder

Two phases.

Phase 1, once per frame pair, 32.8 ms. The frozen pipeline produces the coarse flow and feature maps. These are cached.

Phase 2, once per query batch, 1.25 ms for up to 2,048 points. For a coordinate (x, y) the

decoder samples 3×3 windows from four sources (1/8 context, 1/8 features, 1/16 features, flow-warped frame-1 features), fuses them in a gated MLP, and predicts weights over ten candidates: the nine coarse-flow values around the query plus a bilinear sample.

Output is a convex combination of those candidates. That bounds the prediction inside the locally supported motion range, so the decoder cannot invent displacement its inputs do not support. The head is initialised so the softmax sits on the bilinear candidate, which makes the untrained decoder exactly equivalent to bilinear upsampling (measured: 0.011 px maximum deviation). Training can only move away from that by learning.

2.3 Uncertainty head

An optional eleventh output predicts a per-query error scale b under a Laplace likelihood. It costs no measurable inference time and attaches a confidence value to every correspondence returned.

2.4 Interface

```
state = model.infer_coarse_state(img0, img1)          # once per pair
flow  = model.decode_queries(state, query_coords=q)   # [B,N,2] -> [B,N,2]
flow  = model.decode_queries(state, target_h=H, target_w=W)
flow, b = model.decode_queries(state, query_coords=q, return_uncertainty=True)
```

Coordinates are continuous. $(312.7, 188.2)$ is as valid as $(312, 188)$. Sparse queries reproduce the dense field exactly at matching coordinates, to 0.00057 px.

3. Setup

Evaluation. VKITTI2 Scene18 and Scene20: 1,174 pairs, 460,573,660 valid pixels, per-pixel metrics. Both scenes are excluded from every training set. A guard in the loader enforces it and I verify no overlap at pair and frame level before each run.

Controls. Every training run is generated from one template, so any two differ in exactly one variable. All use seed 1234, batch 16, learning rate 2×10^{-4} on a OneCycle schedule, $\gamma = 0.8$, 4,096 supervision queries per image, refinement of 1×s16 and 8×s8 matching evaluation, and 100,000 steps.

Pre-flight. A nine-check suite runs on CPU and GPU before any job: batch-normalisation frozen, only decoder parameters trainable, zero-initialised decoder equals bilinear, sparse equals dense, uncertainty present in both decode paths, stride-2 fidelity, seed reproducibility.

Hardware. RTX 4060 laptop, fp16, 384×1248 input unless stated.

4. Results

4.1 Accuracy

Model	Training data	EPE (px)	1px (%)	3px (%)	Params
NeuFlow v2	FlyingThings	**2.324**	**77.63**	**89.80**	9.03 M
NeuFlow v3	FlyingChairs	2.500	72.81	87.88	**7.83 M**
NeuFlow v3	+ VKITTI2	2.398	75.74	88.94	7.83 M
NeuFlow v3	+ MPI-Sintel	2.392	75.83	88.98	7.83 M
NeuFlow v3	+ uncertainty head	2.384	76.13	89.02	7.83 M

Every v3 configuration sits above v2. The best, at 2.384 px, is 2.6% worse on mean error and 1.5 points worse on 1-pixel accuracy. The like-for-like comparison is worse still: trained on

FlyingChairs alone, mirroring v2's own training, v3 reaches 2.500 px, which is 7.6% behind.

So the implicit decoder is less accurate than v2's convex upsampler. The reason is identified in Section 6 and is not a training artefact: the decoder's finest input sits at 1/8 resolution while v2's upsampler reads the full-resolution frame.

The ordering is monotonic. Each addition of data helps, and the uncertainty head is best of the four, though the last three differ by less than 0.02 px and I would not read much into their ranking from a single seed.

These numbers replace an earlier set that showed v3 at 2.10 to 2.29 px. Those runs had drifting normalisation statistics in the supposedly frozen stack, worth about 0.25 px. With the statistics genuinely frozen the advantage disappears.

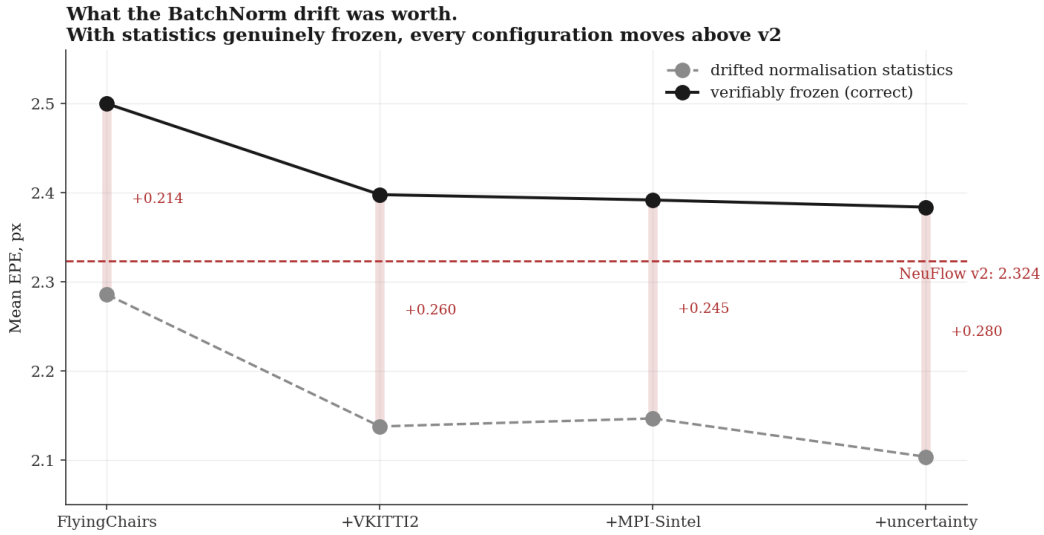

Figure 1: The same four configurations trained with drifting versus verifiably frozen normalisation statistics. The drift was worth roughly 0.25 px and was responsible for v3 appearing to match v2.

4.2 Compute

Mode	Latency	Note
NeuFlow v2, full frame	33.3 ms	its only mode
v3 sparse, first query on a new pair	34.1 ms	level with v2
v3 sparse, repeat query on a cached pair	**1.25 ms**	**27× cheaper**
v3 dense, stride 2	37.1 ms	not the intended mode

The sparse path is a 32.8 ms coarse pass inherited from v2 plus a 1.25 ms decode. Global matching needs whole-image context, so the coarse pass cannot be skipped. It is 87% of a first query, which is why a first query costs what a v2 frame costs.

State reuse is the property that matters. v2 keeps nothing between calls, so asking a second question about the same frame means recomputing everything. v3 answers from cached state in 1.25 ms. For anything that queries a frame more than once (iterative registration, RANSAC refinement, interactive selection, multi-hypothesis tracking) that is a structural difference, not a margin.

Decode latency is 1.254 ms at $N = 800$ and 1.285 ms at $N = 2{,}048$. The decode is bound by kernel launch overhead, not compute, so 2,048 points cost the same as 800.

4.3 Calibrated uncertainty

Predicted scale b	Actual mean error
0.01 to 0.10	0.480 px
0.10 to 0.19	0.896 px
0.19 to 0.39	1.018 px
0.39 to 1.22	1.837 px
above 1.22	7.100 px

Measured over 2,348,000 queries. Actual error rises monotonically across all five bins, a 15× span, Pearson $r = 0.318$. The correlation is not strong but the signal is clearly informative and directly usable: weight correspondences in RANSAC, drop unreliable ones, or send more queries where the model is unsure. v2 outputs flow only and has nothing comparable.

4.4 Interactive tool

A PyQt application loads a video, steps frame by frame, and computes flow for a region the user drags out. Two modes: exact decoding inside the box against a full-frame coarse pass, or a cropped mode running the whole pipeline on the selection so cost scales with the area requested. Both print their latency breakdown live.

5. Flowing only part of the frame

A fast platform cannot afford full-frame flow every frame, so it flows only the region that matters. Three questions follow: what cropping to a region costs, when it pays, and what happens when a new object appears in a frame already being processed.

5.1 What a region costs

Region processed	Area	Latency	Speedup	EPE in region
Full frame	100%	33.3 ms	1.0×	0.657 px
Region, no margin	7.9%	7.8 ms	4.3×	1.089 px
Region + 32 px margin	**13.8%**	**7.6 ms**	**4.4×**	**0.691 px**
Region + 64 px margin	20.1%	8.1 ms	4.1×	0.667 px
Region + 128 px margin	34.2%	9.9 ms	3.4×	0.655 px

Keeping full resolution and processing 14% of the frame costs 0.034 px and runs 4.4× faster. Beyond a 32 px margin nothing improves, so the extra area is wasted.

Cropping applies to any flow network, v2 included. It is a platform technique, not a property of the queryable decoder.

5.2 The margin rule

A crop removes the context global matching uses to find large displacement. With no margin, error rises 0.432 px and 24% of large-motion pixels fail outright. A 32 px margin on 26.6 px mean motion costs 0.034 px and is the knee; beyond it nothing improves. The rule is **margin ≈ expected inter-frame motion ≈ speed / frame rate**, so a faster platform needs a larger margin and saves proportionally less.

5.3 A new object mid-frame

Policy	Cost of the new object	EPE on it
v3, sparse queries (800 pts)	**1.68 ms**	2.10 px
v3, dense ROI	4.36 ms	1.77 px
v2, new crop	7.29 ms	3.60 px
v3, new crop	8.23 ms	3.61 px

Because the coarse pass is cached, the decoder answers a newly appeared object for 1.68 ms. A cropped pipeline must run an entire new pass and is markedly less accurate doing so, having lost the global context. NeuFlow v2 keeps no state between calls and has nothing to answer from. This is the one result in this section specific to the architecture rather than to cropping: answering a question that was not anticipated when the frame arrived.

Two disjoint crops cost more than one full frame, so the policy rule is to crop once or not at all.

6. What v3 is for

The contribution is an operating point, not a leaderboard position. v3 trades 2.6% of mean accuracy and 1.5 points of 1-pixel accuracy for three properties the fixed-resolution design cannot express, from a model 13% smaller:

1. Flow at any continuous coordinate, with sparse output identical to dense.
2. Repeat access at 1.25 ms per query batch against a cached frame, versus a full 33.3 ms recomputation.
3. A calibrated error estimate on every correspondence.

If an application consumes one full dense flow field per frame and nothing else, v2 is the better choice on every axis: more accurate, and 11% faster in that mode. I would say so plainly. v3 earns its place when the consumer picks its own query points, revisits a frame, needs positions between pixels, or wants a confidence value, and when 2.6% of mean accuracy is an acceptable price for those.

7. Limitations

Accuracy. The decoder is less accurate than the upsampler it replaces, by 2.6% on mean error and 1.5 points of 1-pixel accuracy at best, 7.6% on the like-for-like comparison. I know the cause: the decoder's finest input is at 1/8 resolution, so the evidence it sees barely changes inside an 8×8 cell, while v2's upsampler reads the full-resolution frame directly. Adding a Fourier positional encoding of the sub-cell offset changed nothing, which rules out missing positional signal and points at missing high-resolution features. Section 7 item 1 is the proposed fix.

The same limitation means sub-pixel querying is currently closer to interpolation than to genuine sub-pixel inference: queries within one 8×8 cell see nearly identical evidence. The interface is exact and the mechanism is real, but the extra information it can return between pixels is small until the decoder sees full-resolution features.

No embedded measurement. Every latency figure is from a laptop RTX 4060. The edge claim is a design target, not a result.

One evaluation domain. VKITTI2 is synthetic driving footage. It says little about field or survey imagery.

Statistical resolution. One seed per configuration, with checkpoint-to-checkpoint variation up to 0.038 px. Differences below about 0.05 px are not resolved.

8. Next

1. **Full-resolution stem.** Give the decoder a cheap full-resolution feature map, estimated 1 to 2 ms. This attacks the precision limitation directly and makes sub-pixel querying substantive rather than interpolative.
2. **Re-run with frozen statistics**, removing the confound in Section 6 and restoring out-of-domain robustness.
3. **Decode-path optimisation.** Decode cost is flat in the query count. CUDA graph capture, fusing the three samplers that share coordinates, and `torch.compile` (measured at 9% on the coarse pass) together project a first query below v2's latency.
4. **Spring high-resolution evaluation**, testing output above the input sampling rate. Blocked on item 2.
5. **Jetson measurement**, which converts the edge claim into a result or refutes it.
6. **Registration on field imagery**, testing the intended application.

Code, training configurations and the full development record: `github.com/shrirag10/Neuflowv3`. Every number here is reproducible from `scripts/eval_all_runs.py`, `scripts/benchmark_sparse.py` and `scripts/eval_calibration.py`.